Relatório Técnico
Crivo de Eratóstenes com MPI

Trabalho Prático 1 – Computação Paralela 2026/2

19 de outubro de 2026

Abstract

Este relatório documenta a implementação sequencial e a implementação paralela, baseada no modelo mestre–escravo do MPI, do Crivo de Eratóstenes. Também registra a conferência dos requisitos do enunciado na pasta `sieve-of-eratosthenes-mpi`. A implementação e a automação dos experimentos estão presentes; entretanto, nesta entrega não foram encontrados os arquivos de resultados nem uma execução de `gprof`. Assim, os ganhos numéricos de desempenho são descritos metodologicamente, mas não são inventados: as tabelas de speedup e eficiência devem ser preenchidas após a execução dos experimentos em um ambiente com OpenMPI.

1 Conferência dos requisitos

O enunciado solicita uma aplicação C que receba um único inteiro N, use MPI no modelo mestre–escravo, escreva os primos em ordem crescente em `primos.txt`, meça o tempo no mestre sem incluir a escrita do arquivo e apresente um relatório com quatro partes técnicas. A situação encontrada na pasta é:

Requisito	**Situação**	**Evidência**
Entrada de um único N	Atendido	Validação em `src/sequencial.c` e `src/paralelo.c`.
Crivo sequencial	Atendido	`src/sequencial.c`.
Modelo mestre–escravo com MPI	Atendido	Rank 0 distribui intervalos; ranks 1 até $P-1$ processam os blocos em `src/paralelo.c`.
Lista de primos-base compartilhada	Atendido	`MPI_Bcast` em `distribuir_primos_base`.
Recebimento e consolidação dos resultados	Atendido	`MPI_Recv` em `receber_resultados`; os blocos são recebidos por rank.
`primos.txt` em ordem crescente	Atendido no código	`salvar_primos` escreve os blocos na ordem dos ranks, e os intervalos são contíguos e crescentes.
Medição sem escrita do arquivo	Atendido	O tempo termina antes de `salvar_primos` e de `registrar_resultado`.

Requisito	Situação	Evidência
Perfil sequencial com `gprof`	Preparado, sem evidência de execução	O alvo `profile` existe no `Makefile`, mas não há `gmon.out` ou saída do `gprof`.
Experimentos com N e processos variados	Automatizado, sem dados nesta pasta	`scripts/executar_experimentos.sh` e o workflow do GitHub Actions estão presentes; os CSVs são gerados e ignorados pelo Git.
Relatório técnico em PDF	Gerado por este documento	Este arquivo constitui o relatório técnico solicitado.

2 Comparação com a implementação de referência

Também foi analisada a pasta `crivo-de-eratostenes-main/crivo-de-eratostenes-main`, que contém uma implementação anterior do mesmo trabalho. A comparação mostrou que não havia um algoritmo obrigatório ausente no projeto atual. As duas versões usam primos-base até $\sqrt{N}$, dividem $[2, N]$ entre escravos e consolidam os blocos no mestre.

Aspecto	Implementação comparada	Implementação atual
Saída	Escreve diretamente `primos.txt`.	Agora também escreve diretamente `primos.txt`, eliminando a ambiguidade do caminho exigido pelo enunciado.
Coleta MPI	Usa `MPI_Gather` e `MPI_Gatherv`.	Usa `MPI_Send`/`MPI_Recv`; é equivalente para este particionamento e recebe os blocos explicitamente por rank.
Validação	Usa `atoi`, com validação limitada.	Usa `strtol`, verifica limites, quantidade de processos e falhas de alocação.
Tratamento de falhas	Mensagens simples e chamadas diretas a `MPI_Abort`.	Centraliza falhas MPI em `abortar_execucao` e verifica operações de arquivo.
Medição	Começa antes do cálculo dos primos-base.	Começa após a preparação e sincronização, incluindo a distribuição, o processamento e o recebimento, sem incluir a escrita, conforme o enunciado.
Experimentos	Não possui automação de séries ou resumo.	Possui scripts, CSVs, médias, speedup, eficiência e workflow de CI.

Aspecto	Implementação comparada	Implementação atual
Profiling	Contém artefatos binários, mas não uma análise técnica reproduzível.	Possui o alvo reproduzível `make profile N=...`; falta apenas executar e anexar a saída.

A implementação comparada também tem limitações que não devem ser copiadas: `atoi` não distingue entrada inválida de zero, não verifica adequadamente falhas de comunicação e mistura a lista recebida e a lista de envio no `MPI_Gatherv` do mestre. A versão atual preserva a solução mais segura, acrescentando apenas a compatibilidade do nome do arquivo de saída.

3 Perfil de desempenho sequencial

A versão sequencial aloca um vetor booleano de tamanho $N + 1$, marca os múltiplos de cada primo até $\sqrt{N}$ e percorre o vetor para contar os primos. O custo assintótico é $O(N \log \log N)$ para a marcação e $O(N)$ para a contagem, com consumo de memória $O(N)$.

O arquivo `Makefile` disponibiliza:

```
make profile N=<limite>
```

Esse alvo recompila a versão sequencial com `-pg`, executa o programa e chama `gprof`. No estado conferido, o alvo está implementado, mas não há um perfil produzido para identificar quantitativamente as funções mais custosas. Pela estrutura do código, o laço de marcação em `criar_crivo` e a varredura em `contar_primos` são os trechos que devem concentrar o trabalho.

4 Oportunidades de paralelização

O intervalo $[2, N]$ pode ser particionado em blocos contínuos independentes depois que todos os processos conhecem os primos menores ou iguais a $\sqrt{N}$. Para cada primo-base, cada processo marca apenas os múltiplos que caem em seu próprio bloco. A marcação local e a enumeração dos primos não dependem dos demais blocos, o que elimina a necessidade de sincronização durante a fase computacional.

O código paralelo usa a seguinte divisão:

- o mestre calcula os primos-base até $\sqrt{N}$;
- o mestre envia a lista-base com `MPI_Bcast`;
- cada escravo recebe um intervalo contíguo por `MPI_Send`;
- cada escravo executa `criar_crivo_local` e `listar_primos`;
- o mestre recebe primeiro a quantidade e depois a lista de cada escravo;
- a concatenação dos blocos preserva a ordem crescente.

5 Paralelização e comunicação

O processo mestre é o rank 0. A quantidade de escravos é $P-1$, e o intervalo de números é distribuído entre esses escravos. A implementação valida $P \geq 2$, limita N a um inteiro representável e rejeita mais escravos que números disponíveis.

Antes da medição, o programa calcula os primos-base e sincroniza os processos com uma barreira. A medição começa no mestre imediatamente antes do `MPI_Bcast` da lista-base e termina depois do recebimento do último resultado. Portanto, ela inclui a distribuição MPI, o processamento local e a coleta, mas não inclui a escrita de `primos.txt` nem o registro no CSV.

As mensagens usadas são:

Etapa	**Operação**	**Conteúdo**
Lista-base	`MPI_Bcast`	quantidade e valores dos primos
Intervalos	`MPI_Send/MPI_Recv`	início e fim do bloco
Contagem local	`MPI_Send/MPI_Recv`	quantidade de primos
Lista local	`MPI_Send/MPI_Recv`	números primos do bloco

Para execuções normais, a lista é escrita em `primos.txt`, exatamente como solicitado no enunciado. Durante séries grandes, a variável `SALVAR_PRIMOS=0` desativa apenas a escrita, evitando que o custo de I/O e o espaço em disco distorçam os experimentos.

6 Avaliação dos ganhos da paralelização

O script `scripts/executar_experimentos.sh` executa a versão sequencial e as versões MPI para diversos valores de N, quantidades de processos e repetições. A configuração padrão usa 70 limites, cinco repetições e $P \in \{2, 3, 4, 6, 8, 12\}$, totalizando 2450 execuções. Os tempos são registrados em:

- `resultados/sequencial/execucoes.csv`;
- `resultados/paralelo/execucoes.csv`.

Para cada N e P, devem ser calculados:

$$S(N,P) = \frac{T_{\text{seq}}(N)}{T_{\text{par}}(N,P)} \qquad \text{e} \qquad E(N,P) = \frac{S(N,P)}{P}.$$

O script `scripts/gerar_relatorio.sh` já calcula médias, mínimos, máximos, speedup e eficiência a partir dos CSVs. Contudo, a pasta conferida não contém os CSVs gerados, nem `resultados/resumo_execucao.md`. Também não foi possível executar OpenMPI neste ambiente local, pois `docker`, `mpicc` e `mpirun` não estão disponíveis. Consequentemente, não há números de speedup e eficiência que possam ser reportados com validade nesta versão do documento.

Para completar a avaliação, em um ambiente Docker ou Linux com OpenMPI, execute:

```
docker compose build
docker compose run --rm dev bash scripts/executar_experimentos.sh
```

Depois, os valores produzidos em `resultados/resumo_execucao.md` devem ser incorporados a esta seção, juntamente com o hardware e as versões de GCC e MPI registrados pelo script.

7 Conclusão

A conferência confirma que a pasta contém o código-fonte C, o `Makefile`, a configuração Docker, os scripts de experimentos, o gerador de resumo e o workflow de execução MPI. A implementação atende estruturalmente aos requisitos algorítmicos e de comunicação do enunciado.

O ponto pendente não é uma ausência de automação, mas de evidência de execução: os resultados experimentais e o perfil `gprof` não foram encontrados na pasta. Assim, antes da entrega definitiva, é necessário executar o perfil sequencial e a série de experimentos, anexar os CSVs ou o resumo gerado e substituir a ressalva desta seção por uma análise baseada em medidas reais. A apresentação do grupo deve explicar a divisão de dados, as mensagens MPI, a exclusão da escrita do arquivo da medição e a interpretação de speedup e eficiência.

8 Validação executada neste ambiente

Foram executadas as validações que não dependem de uma instalação MPI:

Validação	**Resultado**
`bash -n scripts/executar_experimentos.sh`	Aprovado
`bash -n scripts/gerar_relatorio.sh`	Aprovado
`git diff -check`	Aprovado
Compilação do PDF com `pdflatex` em duas passagens	Aprovado
Compilação C sequencial com o GCC MinGW disponível	Não concluída neste ambiente
Compilação MPI com `mpicc`	Não executada: ferramenta indisponível
Execução MPI com `mpirun`	Não executada: ferramenta indisponível
Instalação do pacote Ubuntu 24.04 via `winget`	Pacote instalado
Inicialização do Ubuntu no WSL	Bloqueada: erro `0x8007019e`
Habilitação/atualização do WSL	Bloqueada: requer administrador, erro `0x80073d28`
Série completa de experimentos	Não executada: requer OpenMPI

O PDF foi gerado com cinco páginas. O compilador LaTeX emitiu avisos de quebra de tabelas longas, mas não apresentou erro fatal e produziu o arquivo de saída. Como o ambiente Windows utilizado não possui `mpicc`, `mpirun` ou Docker. O pacote Ubuntu 24.04 foi instalado via `winget`, mas sua inicialização retornou `0x8007019e`, indicando que o Subsistema do Windows para Linux não está habilitado. A atualização do pacote WSL retornou `0x80073d28`, indicando necessidade de privilégios administrativos. Portanto,

não foi possível obter tempos, contagens de primos, speedups ou eficiências reais nesta sessão.

Depois de habilitar o WSL em um PowerShell executado como administrador, é necessário reiniciar o Windows, abrir o Ubuntu e executar:

```
sudo apt update
sudo apt install -y build-essential binutils libopenmpi-dev openmpi-bin
cd /mnt/c/Users/livia/Desktop/ccomp/2026.2/computacao_paralela/sieve-of-
    eratosthenes-mpi
make
make run-sequential N=100
make run NP=4 N=100
make profile N=1000000
LIMITES="1000 10000 100000 1000000" PROCESSOS="2 4" REPETICOES=3 \
  bash scripts/executar_experimentos.sh
```

Esses comandos produzirão o perfil, os CSVs e `resultados/resumo_execucao.md`, que devem ser incorporados à versão final deste relatório.